Learning User Tastes from 32 Million Ratings with Alternating Least Squares

Yemalin Morel Kpavode [1]

Abstract

[...!] The code is available at https://github.com/yemal22/recom-system

1. Introduction

Recommender systems help users find items, such as movies, books or products, in very large catalogues. In *collaborative filtering*, we do not use any description of the items: we only use the ratings that users gave to items, and we predict the ratings that are missing. The data can be seen as a large matrix of users times items, but this matrix is mostly empty, because each user rates only a very small part of the catalogue.

A successful approach is *matrix factorization*, made popular by the Netflix Prize. Each user and each item is described by a short vector of K hidden "traits", and a rating is predicted by the inner product of the two vectors, plus a user bias and an item bias. The model can be read as a probabilistic model with Gaussian likelihood and Gaussian priors, and it can be trained efficiently with alternating least squares.

In this report we:

- study the MovieLens data and its very uneven (power-law) distribution of ratings per user and per movie;
- derive and implement ALS, first for a model with biases only, then for a model with biases and K-dimensional embeddings;
- train the model on MovieLens 32M with a train/test split *by time*, and study overfitting as a function of K and of the regularization τ;
- use the trained model to recommend movies to a new user, and to find "polarising" movies.

[1]African Institute for Mathematical Sciences (AIMS) South Africa, 6 Melrose Road, Muizenberg 7975, Cape Town, South Africa. Correspondence to: Morel Kpavode <yemalin@aims.ac.za>.



Table 1. MovieLens 32M in numbers. The last three rows are the test RMSE of trivial predictions, with the split by time of Section 5.

Ratings	32,000,204
Users	200,948
Movies (with ratings)	84,432
Density	0.189%
Period	1995–2023
Mean rating (std)	3.54 (1.09)
Ratings per user (median / max)	73 / 33,332
Ratings per movie (median / max)	5 / 102,929
Movies with <100 ratings	85.6%
Test RMSE, global mean	1.058
Test RMSE, user mean	0.959
Test RMSE, movie mean	0.964

2. Data

We use two MovieLens datasets (Harper & Konstan, 2015). Each rating is a tuple (user, movie, rating, timestamp), with ratings from 0.5 to 5 stars in steps of 0.5.

- **ml-latest-small**: 100,836 ratings from 610 users on about 9,700 movies. We use it to develop and test the code (Practicals 1–3).
- **MovieLens 32M**: 32,000,204 ratings from 200,948 users on 84,432 movies (the catalogue lists 87,585 movies, but some have no rating), collected between 1995 and 2023. We use it for the large-scale experiments.

The rest of this section describes MovieLens 32M; Table 1 gives the main numbers.

Sparsity. The rating matrix has about 1.7×10^{10} cells, but only 0.189% of them are observed: 99.8% of the matrix is empty. This is why we never build the full matrix, and why every computation in Section 4 runs over the observed ratings only.

Ratings are mostly positive. The mean rating is 3.54 (standard deviation 1.09), and the most common value is 4 stars, given in about a quarter of all ratings (Figure 2a). Users prefer whole stars: they make up 68.7% of the ratings, although half stars are allowed.

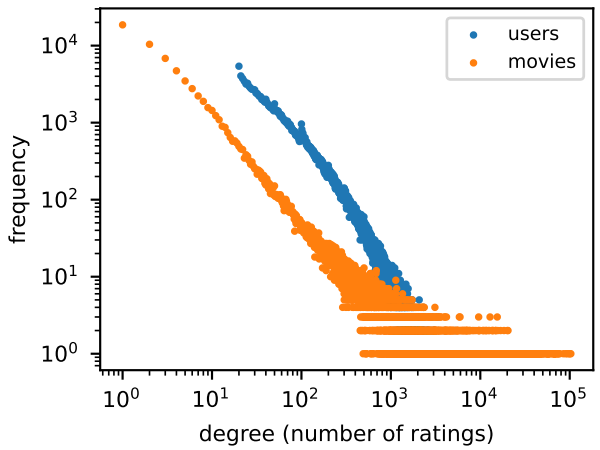


Figure 1. Number of users and movies with a given number of ratings (degree), on a log-log scale (MovieLens 32M).

Power laws and the long tail. The number of ratings per user (the user *degree*) and per movie is very uneven (Figure 1). On a log-log plot both curves are close to straight lines, the sign of a power law. A typical user has 73 ratings (median), but the most active one has 33,332. For movies the contrast is even stronger: the median movie has only 5 ratings, while the most rated one has 102,929. The most popular 1% of the movies receive 54.8% of all ratings, and the top 10% receive 95.7% (Figure 2b). At the other end, 62.1% of the movies have fewer than 10 ratings and 85.6% fewer than 100. This matters for the model: users and especially movies with few ratings have little data to learn a K-dimensional vector, so they rely on the prior, and their vectors are not reliable for recommendations.

Users and movies differ on average. Figure 2c shows the mean rating of each user and each movie (with at least 20 ratings). Both spread widely around the global mean: the standard deviation is 0.49 for users and 0.48 for movies. Some users are generous and some are strict; some movies are liked by almost everyone. This is exactly what the biases $b_m^{(u)}$ and $b_n^{(i)}$ in Section 3 capture, and it explains why a user's or a movie's mean alone already beats the global mean (test RMSE 0.959 and 0.964 against 1.058, Table 1).

Popular movies get higher ratings. The mean rating of a movie grows with its number of ratings (Figure 2d): from 2.91 for movies with fewer than 10 ratings to 3.63 for movies with more than 10,000. Good movies attract more viewers, and people tend to watch what they expect to like. As a result, the item bias is strongly linked to popularity. When we recommend movies, we therefore give the item bias a small weight, so that the most popular movies do not always come first.

Time. The number of ratings per year is far from constant (Figure 2e), with peaks around 2000 and 2015–2016, and the mean rating per year moves between about 3.45 and 3.75. The data therefore changes over time, which is one reason to split train and test by time (Section 5). A surprising fact is that 58.8% of the users gave all their ratings on a *single day*; the median time between a user's first and last rating is 0 days. For these users, the "future" ratings in the test set come from the same session as the training ratings.

Genres. Each movie has one or more genres. Drama and Comedy are by far the most common (Figure 4 in the appendix). Our model does not use genres: it learns its own hidden traits from the ratings only.

3. Model

Let Ω be the set of observed pairs (m, n), where user m rated movie n with rating r_{mn}. We write $\Omega(m)$ for the movies rated by user m, and $\Omega(n)$ for the users who rated movie n.

3.1. Biases only

The simplest model explains a rating with one number per user and one number per movie:

$$\hat{r}_{mn} = b_m^{(u)} + b_n^{(i)}. \tag{1}$$

The user bias $b_m^{(u)}$ says if user m rates generously or not; the item bias $b_n^{(i)}$ says if movie n is liked in general. This model is not personal: all users get the same ranking of movies.

3.2. Biases and embeddings

We add a vector $\mathbf{u}_m \in \mathbb{R}^K$ for each user (their tastes) and a vector $\mathbf{v}_n \in \mathbb{R}^K$ for each movie (its traits):

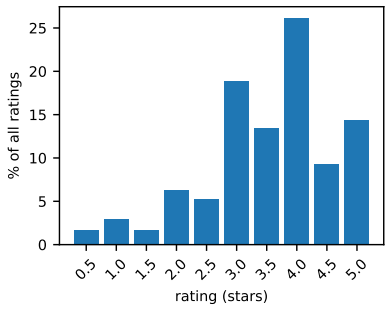
$$\hat{r}_{mn} = \mathbf{u}_m^\top \mathbf{v}_n + b_m^{(u)} + b_n^{(i)}. \tag{2}$$

The term $\mathbf{u}_m^\top \mathbf{v}_n$ is large when the user's tastes match the movie's traits, so predictions become personal. With $K = 0$ we get back the biases-only model.

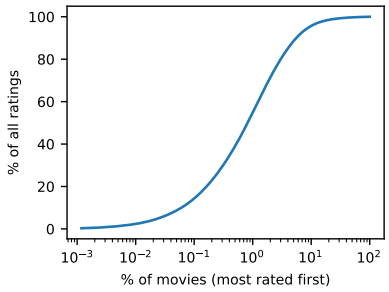
3.3. Probabilistic view and loss

We assume Gaussian noise with precision λ, and Gaussian priors on the vectors (precision τ) and on the biases (precision γ):

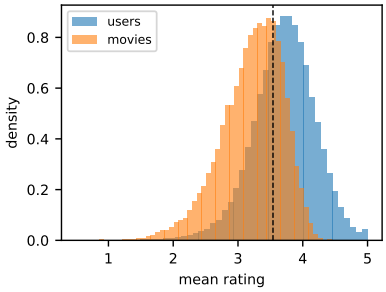
$$\begin{aligned} r_{mn} &\sim \mathcal{N}\big(\hat{r}_{mn},\, \lambda^{-1}\big), \\ \mathbf{u}_m,\ \mathbf{v}_n &\sim \mathcal{N}(\mathbf{0},\, \tau^{-1} I), \\ b_m^{(u)},\ b_n^{(i)} &\sim \mathcal{N}(0,\, \gamma^{-1}). \end{aligned}$$



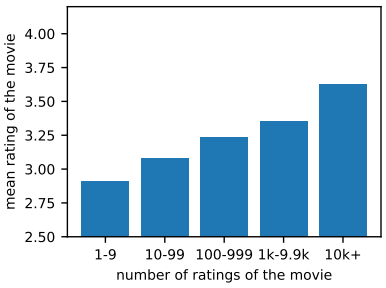
(a) Distribution of the ratings



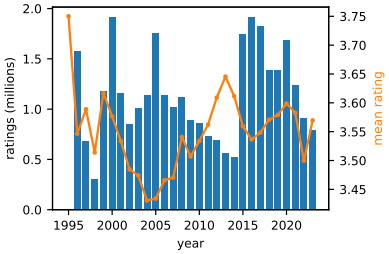
(b) Long tail: share of the ratings



(c) Mean rating of users and movies



(d) Mean rating vs. popularity



(e) Ratings per year and mean rating

Figure 2. Exploratory analysis of MovieLens 32M. (c) only uses users and movies with at least 20 ratings; the dashed line is the global mean.

The maximum a posteriori (MAP) estimate minimises the negative log posterior, up to a constant:

$$E = \frac{\lambda}{2} \sum_{(m,n)\in\Omega} \left(r_{mn} - \hat{r}_{mn}\right)^2 + \frac{\tau}{2}\Big(\sum_m \|\mathbf{u}_m\|^2 + \sum_n \|\mathbf{v}_n\|^2\Big) + \frac{\gamma}{2}\Big(\sum_m (b_m^{(u)})^2 + \sum_n (b_n^{(i)})^2\Big). \tag{3}$$

with $\hat{r}_{mn}$ from (2). Only the ratios τ/λ and γ/λ change the minimiser, so we fix $\lambda = 1$.

4. Training with Alternating Least Squares

The loss (3) is not convex in all parameters together, because of the product $\mathbf{u}_m^\top \mathbf{v}_n$. But if all movie parameters are fixed, E is a quadratic function of the parameters of one user, and we can minimise it exactly. ALS alternates between the users and the movies.

User bias. Setting $\partial E / \partial b_m^{(u)} = 0$ with everything else fixed gives (all sums are over $n \in \Omega(m)$)

$$b_m^{(u)} = \frac{\lambda \sum_n \left(r_{mn} - \mathbf{u}_m^\top \mathbf{v}_n - b_n^{(i)}\right)}{\lambda\,|\Omega(m)| + \gamma}. \tag{4}$$

It is the average residual of the user, shrunk towards 0 by γ: a user with few ratings is shrunk more.

User vector. Setting $\nabla_{\mathbf{u}_m} E = 0$ gives a small $K \times K$ linear system, $A_m \mathbf{u}_m = \mathbf{c}_m$, with

$$A_m = \lambda \sum_n \mathbf{v}_n \mathbf{v}_n^\top + \tau I, \qquad \mathbf{c}_m = \lambda \sum_n \left(r_{mn} - b_m^{(u)} - b_n^{(i)}\right)\mathbf{v}_n. \tag{5}$$

The term τI makes the matrix invertible, even for a user with fewer than K ratings. The movie updates are the same, with the roles of users and movies swapped. Each update gives the exact minimum for its block of parameters, so the loss can only go down at every iteration; we use this as a check of the implementation.

Initialisation. The vectors must not start at 0: if all $\mathbf{v}_n = \mathbf{0}$, then $\mathbf{c}_m = \mathbf{0}$ in (5), so all $\mathbf{u}_m$ stay 0, and the model never learns. We start from small random numbers.

Cost. One iteration costs $O(|\Omega| K^2)$ to build the matrices and $O((M+N)K^3)$ to solve the systems. Since K is small, the cost grows linearly with the number of ratings.

Algorithm 1 ALS for biases and embeddings

Input: Ω, K, λ, τ, γ, iterations T
Initialise $\mathbf{u}_m, \mathbf{v}_n$ with small random numbers; biases $= 0$
for $t = 1$ **to** T **do**
  for each user m **do**
    update $b_m^{(u)}$ (4), then $\mathbf{u}_m$ (5)
  end for
  for each movie n **do**
    update $b_n^{(i)}$, then $\mathbf{v}_n$ (roles swapped)
  end for
  compute the loss E and the train/test RMSE
end for

Table 2. Train and test RMSE on ml-latest-small (random 10% test split per user; 10 ALS iterations for biases only, 20 for $K = 10$).

Model	Train	Test
Biases only	0.783	**0.875**
Biases + $K = 10$	**0.555**	0.893

5. Experimental Setup

Train/test split. A recommender must predict the *future*, so a random split would be too optimistic. For each user, we sort the ratings by timestamp and keep the newest 10% for the test set. Since every user in MovieLens 32M has at least 20 ratings, every user has at least 2 test ratings. This gives 28,885,713 training and 3,114,491 test ratings. Only 0.28% of the test ratings are on movies that never appear in the training set; for those, the model can only use the user's parameters.

Metric. We report the root mean squared error on the train and test sets,

$$\mathrm{RMSE} = \sqrt{\frac{1}{|\Omega_{\text{test}}|} \sum_{(m,n)\in\Omega_{\text{test}}} \left(r_{mn} - \hat{r}_{mn}\right)^2}.$$

Hyperparameters. Unless stated otherwise, $\lambda = 1$, $\gamma = 0.1$, $\tau = 5$, $K = 10$, and 10 ALS iterations.

6. Results

6.1. Development on ml-latest-small

We first built and checked the method on the small dataset (100,836 ratings), with $\lambda = 1$, $\gamma = 0.1$ and, for the full model, $K = 10$ and $\tau = 5$. Table 2 and Figure 3 give a global view.

ALS behaves as expected. For both models, the loss goes down at every iteration, which is the check described in Section 4. The biases-only model converges almost at once:

Table 3. Train and test RMSE on MovieLens 32M (split by time).

Model	Train RMSE	Test RMSE
Global mean	–	1.058
User mean	–	0.959
Movie mean	–	0.964
Biases only	–	–
Biases + $K = 10$	–	–
Biases + $K = 20$	–	–

after 2 iterations the RMSE does not change any more, because each bias depends only weakly on the others. The model with embeddings needs about 10 iterations, since users and movies have to agree on the meaning of the K directions.

Embeddings fit the training data much better. With $K = 10$, the train RMSE falls from 0.783 to 0.555: the vectors capture personal tastes that the biases cannot.

… but they overfit on small data. The test RMSE does *not* improve: it goes from 0.875 to 0.893. With only about 165 ratings per user on average, 10 numbers per user and per movie are too many for the regularization $\tau = 5$: the vectors partly memorise the training ratings. This gap between train and test is the main motivation for the next experiments: a larger dataset (MovieLens 32M) and a study of K and τ.

6.2. Biases only vs. biases and embeddings

6.3. Overfitting

6.4. Recommendations for a new user

6.5. Polarising movies

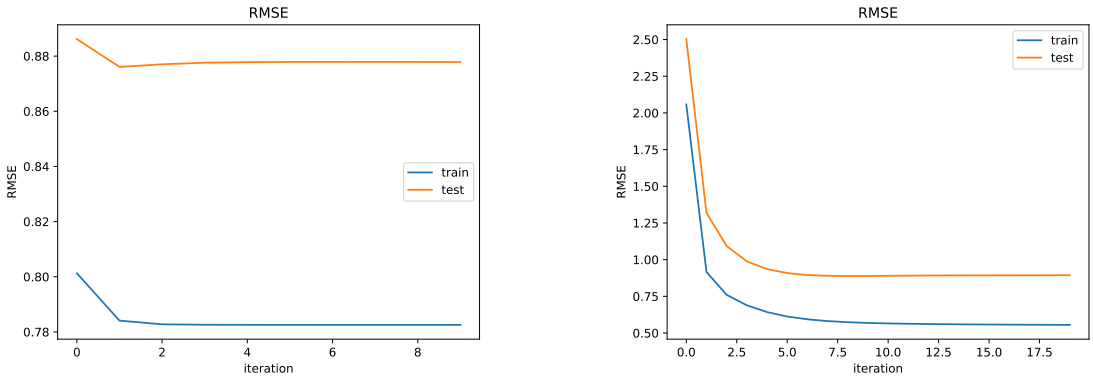
7. Conclusion

Software and Data

The code is available at https://github.com/yemal22/recom-system.

References

Harper, F. M. and Konstan, J. A. The MovieLens datasets: History and context. *ACM Transactions on Interactive Intelligent Systems*, 5(4):19:1–19:19, 2015.



(a) Biases only

(b) Biases + embeddings, $K = 10$

Figure 3. Train and test RMSE during ALS on ml-latest-small.

A. Additional Data Analysis

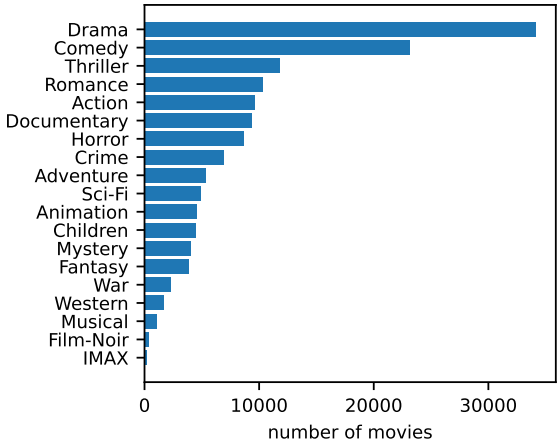


Figure 4. Number of movies per genre in MovieLens 32M (a movie can have several genres).